Explainable AI-Driven Autonomous Self-Healing Cloud Management Platform

Rahul TS
Department of Computer Science and Engineering
School of Computer Science and Technology
Karunya Institute of Technology and Science
Coimbatore, Tamil Nadu, India
rahult23@karunya.edu.in

***Abstract*—Autonomous cloud resilience demands proactive management platforms that forecast emerging resource anomalies, diagnose underlying drivers, and trigger remediations before critical service disruption occurs. This paper presents an explainable AI-driven autonomous cloud management platform that integrates ten sequentially connected processing stages: telemetry ingestion, anomaly detection, temporal buffer management, temporal feature extraction, multi-model failure-risk prediction, deterministic model-coefficient attribution, likely-failure-category inference, autonomous decision logic, simulated recovery, and post-recovery verification. The system is implemented as a deployable research prototype and evaluated on a controlled synthetic telemetry dataset covering eight distinct scenario profiles: two normal baseline profiles (stable and variable) and six failure scenarios (CPU spike, gradual CPU failure, memory leak, disk gradual exhaustion, network degradation, and combined gradual failure). Isolation Forest anomaly detection achieves precision 0.704, recall 0.826, and F1 0.760 on snapshot telemetry. Temporal Logistic Regression achieves mean F1 0.692 ($\pm$ 0.036) and ROC-AUC 0.813 ($\pm$ 0.021) across five random seeds, substantially outperforming both a majority-class baseline (F1 0.514) and a current-value threshold baseline (F1 0.156). Among 18 evaluated failure sequences, the model issued at least one high-risk prediction before labelled failure onset with a mean lead time of 18.06 timesteps. The XAI attribution layer computes exact coefficient-based contributions for each of 45 temporal features, enabling transparent identification of which telemetry patterns drive each prediction. A post-recovery verification mechanism re-evaluates simulated recovered telemetry using the same model, returning RECOVERED, NOT_RECOVERED, or INCONCLUSIVE verdicts. All experiments are conducted on synthetic data; real-world deployment validation is identified as future work.**

***Index Terms*—cloud monitoring, anomaly detection, temporal feature engineering, explainable AI, coefficient attribution, AIOps, self-healing, failure prediction, autonomous remediation**

I. Introduction

Distributed cloud architectures emit continuous multidimensional telemetry spanning compute, memory, storage, and networking interfaces. Under operational scale, the sheer throughput of telemetry metrics rapidly overwhelms manual operations, while the window between initial performance degradation and customer-impacting outages frequently collapses below human triage latency [1]. To bridge this operational gap, Artificial Intelligence for IT Operations (AIOps) architectures have emerged to automate fault identification, root-cause diagnostics, and targeted remediation [2], [3].

Prior literature in cloud reliability typically isolates individual lifecycle phases. Unsupervised anomaly detectors—including Isolation Forests, log-based recurrent architectures [4], [5], and variational autoencoders [6], [7]—attain strong detection accuracy over specific metric streams but remain decoupled from active remediation. Conversely, autonomous self-healing engines [8], [9] trigger reactive mitigation upon static threshold breaches, lacking forward-looking temporal prediction and interpretability. Root-cause analysis (RCA) techniques [10]–[12] reconstruct causal dependencies across distributed traces, yet they require heavy runtime instrumentation and omit closed-loop post-recovery verification.

Consequently, modern operations suffer from fragmentation: disparate specialized modules that are rarely evaluated as a cohesive, end-to-end operational pipeline from live ingestion to post-remediation verification. This investigation resolves this limitation by constructing, validating, and benchmarking a unified platform that connects all operational stages into a reproducible research prototype.

II. Related Work and Research Gap

A. Cloud Anomaly Detection

Early cloud monitoring focused on static snapshot telemetry. He et al. [13] demonstrated through an empirical benchmarking of classical classifiers on system logs that linear and decision-tree baselines remain competitive against complex architectures on specific subsets. To capture multivariate temporal dynamics in continuous key performance indicators (KPIs), Xu et al. [7] developed Donut using variational autoencoders for seasonal metrics, attaining F1 scores between 0.75 and 0.90. Su et al. [6] introduced OmniAnomaly, leveraging stochastic recurrent units to model temporal metric distributions (0.89 F1 on Server Machine Datasets), while Li et al. [14] formulated InterFusion to correlate macroscopic observations with hidden inter-metric dependencies. Although these models detect anomalous deviations effectively, they generate black-box anomaly scalars that neither isolate specific contributory telemetry features nor guide automated remediation.

Sequential log and trace monitoring extends point-in-time checks across discrete execution flows. Du et al. [4] developed DeepLog to model normal log sequences through LSTM layers (0.96 F1 on HDFS logs), and Meng et al. [5] introduced

LogAnomaly to integrate semantic template embeddings with sequential classification (0.98 F1 on BGL logs). In microservice environments, Nedelkoski et al. [15] applied sequence-based learning across distributed trace graphs, achieving 0.95 F1. Despite their diagnostic precision, these methods cannot explain the underlying parametric drivers in continuous telemetry or translate alerts into verifiable remediation workflows.

B. AIOps and Intelligent Operations

Systematic taxonomies by Lin et al. [1] at Microsoft Azure established anomaly detection, root-cause identification, and automated remediation as the core triad of modern AIOps. Bogatinovski et al. [2] and Niu et al. [3] surveyed deep learning techniques across cloud operations, highlighting the division between isolated diagnostic tools and production execution pipelines. In a broader evaluation of monitoring systems, Islam et al. [16] observed that while passive anomaly filtering is mature, proactive failure forecasting and closed-loop self-healing frameworks remain under-represented in published experimental literature.

C. Explainability for Operational Systems

Post-hoc explainability methods provide local interpretations for opaque models. Ribeiro et al. [17] proposed LIME using local linear surrogates, whereas Lundberg and Lee [18] formulated SHAP using cooperative game theory to allocate Shapley values. In operational cloud environments, however, both frameworks incur sampling overhead and introduce approximation variance. Li et al. [19] categorized XAI for anomaly detection into post-hoc, model-intrinsic, and feature-based classes. In contrast to sampling approximations, linear models allow exact, closed-form coefficient attribution (scaled feature $\times$ weight), delivering deterministic explainability with minimal computational cost.

D. Root Cause Analysis

Structural diagnostic frameworks isolate faulty components through dependency topology. Wang et al. [10] designed CloudRanger using dynamic service graphs and random-walk traversal, while Chen et al. [12] introduced CauseInfer to diagnose multi-tier performance degradations (82% accuracy). Weng et al. [11] applied constraint-based PC algorithms to discover causal graphs directly from metric streams. While effective, these graph-based methods depend on heavy distributed tracing agents. This work adopts a lighter, non-invasive alternative: aggregating model attribution scores across resource domains (CPU, Memory, Disk, Network) to infer the dominant failure category without requiring tracing infrastructure.

E. Self-Healing and Automated Remediation

Closed-loop remediation has progressed from static thresholds toward adaptive controls. Samir et al. [8] demonstrated fuzzy-logic microservice remediation, cutting mean time to repair by 60% but relying on static heuristic rules without predictive forecasting. Gan et al. [20] designed Seer, using hardware performance counters to preemptively forecast microservice quality-of-service violations with 91% accuracy, yet without explainable attribution or root-cause localization. Zhang et al. [9] analyzed automated incident remediation through historical guide recommendations, yet their approach lacks deterministic model-level explanations and does not verify post-recovery efficacy using the predictive classifier.

F. Research Gap and Open Challenges

The review in Section II reveals that the cloud management literature addresses individual pipeline stages well but rarely evaluates them as a unified end-to-end workflow. Specifically:

1) Detection methods do not explain which features drove the alert.
2) XAI methods are applied to detection models but not connected to recovery actions.
3) Self-healing systems do not use temporal prediction and do not verify recovery with the same model.
4) Survey papers on Artificial Intelligence for IT Operations (AIOps) repeatedly identify system integration as an open challenge, yet they rarely provide integrated experimental evaluations.

This work addresses these gaps by implementing and evaluating an integrated prototype that connects telemetry ingestion, temporal prediction, deterministic XAI attribution, failure-category inference, decision logic, simulated recovery, and model-based verification in a single system. While this system does not claim to be the first to combine cloud monitoring with self-healing, nor does it assert superiority over deep learning anomaly detectors, it provides a comprehensive end-to-end framework demonstrating a complete, interpretable, reproducible workflow in a research prototype setting.

G. Research Questions and Contributions

This investigation evaluates five core research questions: **RQ1** (Detection Fidelity): Can ML anomaly detection identify resource failures across synthetic telemetry? **RQ2** (Early Warning): Do temporal rolling features improve lead time over snapshot threshold alarms? **RQ3** (Deterministic XAI): Can closed-form coefficient attribution reliably isolate influential telemetry patterns? **RQ4** (Remediation Mapping): Can attribution evidence accurately guide automated recovery actions? **RQ5** (Closed-Loop Verification): Can model re-evaluation provide an honest, conservative post-healing verification signal?

The primary technical contributions are: (1) an end-to-end prototype uniting all ten lifecycle stages in a callable API; (2) a 45-feature temporal feature extraction layer strictly eliminating lookahead leakage; (3) a rigorous 5-seed comparative evaluation across sequence-isolated splits; (4) a closed-form, deterministic XAI attribution engine requiring zero sampling approximations; and (5) a closed-loop verification protocol that re-tests remediated states against the original predictive classifier.

III. Proposed System Architecture and Methodology

The proposed methodology connects telemetry ingestion, anomaly detection, temporal feature extraction, multi-model failure-risk prediction, deterministic model-coefficient attribution, likely-failure-category inference, autonomous decision logic, and post-recovery verification as a unified sequential pipeline (Fig. 1).

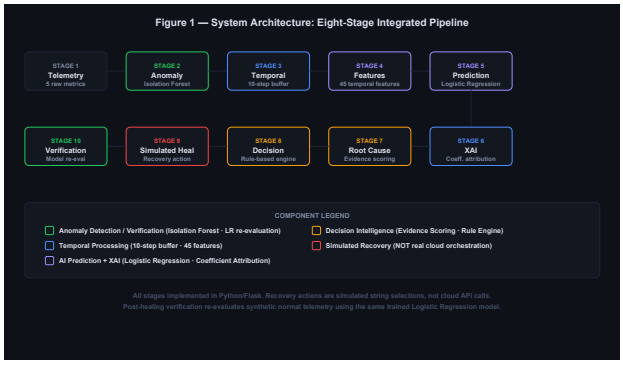


Fig. 1. Explainable AI-Driven Cloud Management System Architecture.

The platform comprises ten sequential processing stages: (1) **Telemetry Ingestion** simulates multi-resource metric streams across CPU, memory, storage, and network interfaces; (2) **Anomaly Detection** performs snapshot point-in-time screening via unsupervised Isolation Forest; (3) **Temporal Buffer Management** maintains a 10-step sliding window of historical context; (4) **Feature Extraction** derives 45 leakage-free rolling statistics and linear trends; (5) **Failure Risk Prediction** estimates forward-looking failure probability using regularized Logistic Regression; (6) **XAI Attribution** derives exact signed feature contributions via model weights; (7) **Failure Categorization** aggregates attribution evidence across resource domains; (8) **Decision Logic** maps dominant failure categories to targeted recovery actions; (9) **Simulated Recovery** safely logs remediation directives without destructive cloud calls; and (10) **Post-Recovery Verification** re-evaluates simulated restored telemetry using the identical predictive classifier.

A. Temporal Feature Engineering

A leak-free temporal feature set comprising 45 features is constructed from each 10-step trailing window. For each of the five raw metrics, nine features are computed: current value, 5-step mean, 10-step mean, 5-step delta, 10-step delta, 5-step slope (via linear regression), 10-step slope, 5-step standard deviation, and 10-step standard deviation. The feature extraction module strictly utilizes trailing observations, ensuring no access to future values (thereby preventing data leakage). StandardScaler is fit only on training rows and applied consistently during live inference.

B. Predictive Modeling and Target Formulation

Logistic Regression was selected for the integrated system with configuration: C=1.0, max_iter=2000, random_state=42, class_weight='balanced'. The prediction target is strictly forward-looking and binary: a label of 1 denotes that an active failure occurs within the subsequent 10 future timesteps ($t+1$ to $t+10$), strictly excluding the current observation at timestep t to avoid target circularity, and 0 otherwise.

The model is trained at server initialisation on the Phase 3C seed-42 training split (4,920 rows from 120 sequences). The fitted scaler and model are used for all live inference. Logistic Regression is selected over Random Forest and Gradient Boosting not because it achieves the highest aggregate performance — all three achieve similar F1 and AUC within measurement noise — but because its coefficient-based attribution is mathematically exact and requires no post-hoc approximation method.

C. Explainable AI and Failure Categorization

1) XAI Attribution: For each live prediction, the contribution of feature i is computed according to (1):

$$\text{contribution}_i = \text{scaled_value}_i \times \text{coefficient}_i \tag{1}$$

where contribution_i denotes the signed contribution of feature i to the predicted log-odds of failure, $\text{scaled_value}_i = (x_i - \mu_i)/\sigma_i$ is the z-score normalized feature value computed via the training-set StandardScaler, and coefficient_i is the corresponding learned Logistic Regression weight. A positive contribution increases the predicted failure probability toward the high-risk threshold, whereas a negative contribution decreases it. The top 5 features by absolute contribution are reported for each prediction. This method is deterministic and computationally inexpensive.

2) Likely Failure Category Inference: Evidence scores for four resource categories (CPU, Memory, Disk, Network) are computed by summing absolute contributions of features belonging to each resource. The category with the highest summed evidence is reported as the likely failure category. This is a heuristic inference from model evidence, not a causal determination.

D. Decision and Simulated Self-Healing

When predicted failure risk reaches the actionable threshold, the autonomous decision engine maps the inferred likely failure category to a simulated recovery procedure, as summarized in Table I.

TABLE I
Likely Failure Category to Simulated Recovery Action Mapping

Likely Failure Category	Simulated Recovery Action
CPU Spike	Terminate / throttle high-CPU processes
Memory Leak	Restart affected service / reclaim memory
Disk Exhaustion	Clear logs / archive data / expand storage
Network Degradation	Check / restart network service / reroute
Combined Failure	Controlled recovery sequence + operator alert

E. Post-Healing Verification Protocol

When predicted failure risk triggers a HIGH or CRITICAL alert, the system executes an automated verification cycle: (1) pre-remediation telemetry and risk states are recorded; (2) the decision engine dispatches the mapped recovery action (simulated string selection without destructive infrastructure calls); (3) a 10-step stable telemetry sequence is synthesized as a recovery proxy; (4) the identical trained Logistic Regression classifier re-evaluates the post-recovery window; and (5) a tripartite verification verdict is determined: **RECOVERED** if post-healing risk drops to LOW (≤ 0.25), **NOT_RECOVERED** if risk remains elevated (≥ 0.50), or **INCONCLUSIVE** if risk hovers in the intermediate range ($0.25 < \text{risk} < 0.50$). The sliding-window buffer is subsequently flushed, returning the runtime loop to the WARMING_UP state.

IV. Experimental Setup

A. Dataset Configuration and Splits

The evaluation dataset encompasses eight distinct scenario types: two normal baseline profiles (`normal_stable` and `normal_variable`) and six failure scenarios (`cpu_sudden_failure`, `cpu_gradual_failure`, `memory_gradual_leak`, `disk_gradual_exhaustion`, `network_degradation`, and `combined_gradual_failure`). Across these eight scenarios, a total of 180 sequences were generated and partitioned into 120 training sequences (4,920 supervised rows), 30 validation sequences (1,230 supervised rows), and 30 test sequences (1,230 supervised rows), yielding exactly 7,380 supervised rows under a 10-step trailing window and 10-step prediction horizon censoring ($60 - 10 - 10 + 1 = 41$ rows per 60-step sequence), as summarized in Table II.

TABLE II
Dataset Splits and Supervised Row Counts

Split	Sequences	Supervised Rows
Training	120	4,920
Validation	30	1,230
Test	30	1,230
Total	**180**	**7,380**

Sequence-level splitting ensures no temporal window from a test sequence is visible during training. Zero sequence overlap is verified by an automated audit (evaluation_audit.json).

Positive label definition: A timestep t is labelled positive ($y_t = 1$) if an active failure is present in any of the next 10 future observations ($t + 1 \leq t' \leq t + 10$). The current observation at timestep t is strictly excluded from the target calculation, ensuring that the model is trained exclusively on advance predictive indicators rather than instantaneous state detection. Positive samples constitute approximately 52% of the dataset under this early-warning formulation.

B. Anomaly Detection Evaluation (Phase 2)

Isolation Forest is evaluated on snapshot telemetry. Baselines include a fixed-threshold classifier (alarm if any metric exceeds 80%) evaluated on identical data.

C. Temporal Predictive Evaluation (Phase 3)

Five model architectures were benchmarked across five independent random seeds {42, 123, 456, 789, 2026}: (1) majority-class baseline; (2) current-value threshold baseline (flagging when any metric exceeds 80%); (3) balanced Logistic Regression ($C = 1.0$, max_iter $= 2000$); (4) balanced Random Forest (100 estimators); and (5) Gradient Boosting (100 estimators). Performance was quantified via accuracy, precision, recall, F1-score, ROC-AUC, and PR-AUC.

D. Early-Warning Horizon Evaluation

Among test sequences containing labelled failure regions, the number of timesteps by which the model issues its first HIGH or CRITICAL prediction prior to the labelled failure onset is measured.

E. Explainability Evaluation Protocol

Four representative XAI case studies are documented from test sequences spanning different scenarios. For each case, the top 5 contributing features, their directions, and the evidence scores are recorded.

V. Experimental Results and Analysis

A. Detection Performance (Phase 2)

The baseline snapshot anomaly detection performance of Isolation Forest compared to the threshold baseline is shown in Table III.

TABLE III
Anomaly Detection: Isolation Forest vs Threshold Baseline

Method	Accuracy	Precision	Recall	F1	FPR
Threshold Baseline	0.903	1.000	0.435	0.606	0.000
Isolation Forest	0.910	0.704	0.826	0.760	0.073

The Isolation Forest achieves substantially higher recall than the threshold baseline (0.826 vs 0.435), correctly identifying most failure events at the cost of a 7.3% false positive rate. The threshold baseline produces zero false positives but misses 56.5% of failures — particularly gradual-onset scenarios where no metric exceeds 80% until late in the failure trajectory. Per-scenario detection analysis shows that CPU spike (F1 0.823), memory leak (F1 0.870), and network degradation (F1 1.00) are detected reliably. Disk gradual exhaustion achieves only F1 0.700 at snapshot level, reflecting the slow accumulation rate relative to the 80% threshold.

B. Temporal Predictive Performance (Phase 3)

The performance of the temporal predictive models aggregated over five independent random seeds is presented in Table IV and illustrated in Fig. 2.

All three ML classifiers substantially outperform both baselines. The differences among LR, RF, and GB are small

TABLE IV
TEMPORAL MODEL COMPARISON (5-SEED MEAN ± STD)

Model	F1	ROC-AUC	PR-AUC	Precision	Recall
Majority Baseline	0.514 ± 0.259	—	—	0.380 ± 0.193	0.800 ± 0.400
Threshold Baseline	0.156 ± 0.054	—	—	0.997 ± 0.006	0.086 ± 0.031
Logistic Regression	**0.692** ± **0.036**	0.813 ± 0.021	0.840 ± 0.027	0.730 ± 0.049	0.661 ± 0.050
Random Forest	**0.696** ± **0.021**	0.819 ± 0.005	0.847 ± 0.017	0.805 ± 0.047	0.620 ± 0.057
Gradient Boosting	**0.699** ± **0.030**	0.826 ± 0.013	0.852 ± 0.025	0.747 ± 0.137	0.701 ± 0.127

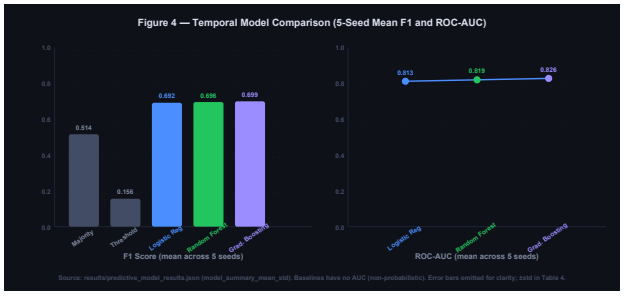


Fig. 2. Temporal Model Comparison (5-Seed Mean F1 and ROC-AUC).

relative to within-seed variance. GB achieves the highest mean F1 (0.699) and AUC (0.826) but also the highest variance in recall (± 0.127). Logistic Regression achieves the most stable performance (F1 std ± 0.036). At seed 42 specifically: LR achieves F1 0.727, ROC-AUC 0.830; RF F1 0.718, ROC-AUC 0.819; GB F1 0.725, ROC-AUC 0.812.

Threshold baseline analysis: The current-value threshold achieves precision 0.997 but recall 0.086 — it correctly classifies near-zero events when a metric exceeds 80%, but misses the vast majority of failure timesteps because gradual failures build slowly below the threshold. This confirms that temporal feature-based prediction provides substantially better early-warning sensitivity.

C. Per-Scenario Predictive Performance

Table V reports the breakdown of classification performance across each evaluated failure scenario for Logistic Regression on the test split.

TABLE V
PER-SCENARIO PREDICTIVE PERFORMANCE (LR, SEED 42, TEST SET)

Scenario	Support	Prec	Recall	F1	TN	FP	FN	TP
normal_stable	205	0.00	0.00	0.00	132	73	0	0
normal_variable	164	0.00	0.00	0.00	164	0	0	0
cpu_sudden_fail	41	0.529	0.529	0.529	16	8	8	9
cpu_gradual_fail	164	0.817	0.527	0.641	41	13	52	58
mem_gradual_leak	41	1.000	0.649	0.787	4	0	13	24
disk_gradual_exh	246	0.952	0.791	0.864	12	9	47	178
network_degrade	123	0.635	0.783	0.702	36	27	13	47
combined_gradual	246	0.820	0.755	0.786	56	27	40	123

Several operational dynamics emerge from the per-scenario breakdown (Table V): (1) **Normal baseline profiles** produce zero false negatives alongside moderate false alarms (73/205 on `normal_stable`), attributable to sensitivity-oriented calibration (`class_weight='balanced'`). (2) **Sudden failures** (`cpu_sudden_fail`) attain the lowest F1 (0.529), illustrating the predictive boundary when physical precursor signals are absent. (3) **Gradual failure modes** (`disk_gradual_exh`, `mem_gradual_leak`, `combined_gradual`) achieve high predictive utility (F1 scores between 0.786 and 0.864), confirming that multi-window rolling trends reliably capture accumulating exhaustion. (4) **Memory leaks** attain 1.000 precision, ensuring that automated mitigations are never falsely triggered, while recall (0.649) highlights opportunity for expanded early-trajectory capture.

D. Early-Warning Performance

The advance warning horizon achieved across all evaluated failure sequences is detailed in Table VI.

TABLE VI
EARLY-WARNING STATISTICS

Metric	Value
Failure sequences evaluated	18
Mean lead time (timesteps)	18.06
Median lead time	16.0
Minimum lead time	4
Maximum lead time	33

The model issued at least one HIGH or CRITICAL prediction before the labelled failure onset in all 18 evaluated failure sequences. Mean lead time of 18.06 timesteps represents substantial advance warning. The minimum of 4 timesteps occurred on sudden-failure scenarios, consistent with the per-scenario results.

E. XAI Attribution Case Studies

Table VII summarizes representative XAI attribution case studies, with corresponding feature contributions visualized in Fig. 3.

TABLE VII
REPRESENTATIVE XAI CASE STUDIES

Case	Scenario	Risk	Top Feature	Contrib	Dir	Likely Cat
test_1	disk_gradual	0.591	Mem 10-step avg	-0.999	↓	Combined
test_4	combined_gradual	0.537	CPU 10-step var	-0.422	↓	Combined
test_6	network_degrade	0.534	Mem 10-step var	+0.416	↑	Combined
test_15	cpu_sudden	0.686	Mem 10-step avg	+0.917	↑	Memory Leak

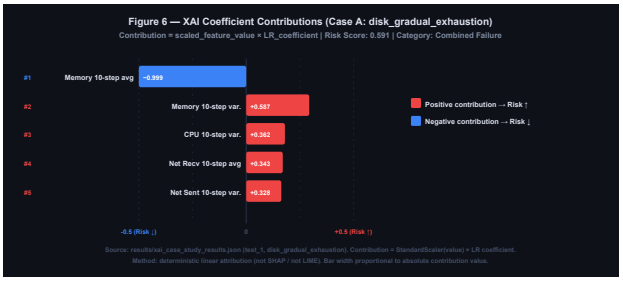


Fig. 3. XAI Feature Contributions Across Representative Evaluation Cases.

The XAI attribution shows that variability features (standard deviation) and slope features frequently dominate contributions in failure scenarios, reflecting that gradual failures increase temporal variation and trend steepness before absolute values reach alarm thresholds. Memory mean features show negative contributions in some cases, meaning that the model interprets high mean memory values as evidence against the specific failure label — a reasonable behaviour when the

model has learned that combined failures show different multi-resource patterns.

F. Integrated Recovery Verification

The integrated system was evaluated across all six failure scenarios. For the network degradation scenario, the verification sequence and resulting risk transition are reported in Table VIII and depicted in Fig. 4:

TABLE VIII
POST-RECOVERY VERIFICATION (NETWORK DEGRADATION)

Stage	Value
Pre-healing risk	CRITICAL (0.884)
Simulated recovery	Controlled recovery sequence + alert
Post-healing risk	MEDIUM (0.447)
Verification verdict	INCONCLUSIVE

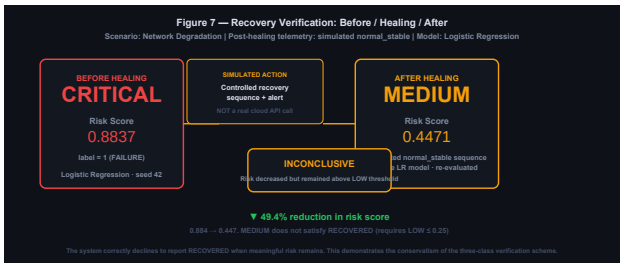


Fig. 4. Post-Recovery Verification Flow.

Risk decreased substantially (from 0.884 to 0.447) but did not reach LOW, producing INCONCLUSIVE. This is the expected and honest result: the model verifies against synthetic normal telemetry, and the specific synthetic sequence generated may include moderate variance that keeps the score above the LOW threshold. Healing actions are simulated string selections dispatched by the rule-based engine, not executable cloud orchestration API calls. Post-healing telemetry is synthetically generated normal telemetry used as an algorithmic evaluation proxy, not physical cloud recovery. The system correctly avoids falsely claiming RECOVERED.

VI. DISCUSSION

A. Temporal Early Warning and Sensitivity Trade-offs

The experimental findings confirm that rolling temporal features provide substantially superior early-warning sensitivity compared with snapshot threshold monitoring. The threshold baseline achieves recall of only 8.6% (mean across seeds) because gradual degradations accumulate beneath static alarm boundaries until late in the failure progression. Conversely, Logistic Regression trained on multi-window features attains 66.1% mean recall with an average lead time of 18.06 timesteps. In production deployments, however, this high sensitivity involves a documented operational trade-off: false positive alerts during benign bursty periods (`normal_stable`), which require threshold dampening or alert coalescing.

B. Interpretability Guarantees in Autonomous Operations

While ensemble architectures (Random Forest and Gradient Boosting) attained marginally higher mean F1 scores (0.696 and 0.699 vs 0.692), Logistic Regression was deliberately selected as the authoritative operational model. In mission-critical cloud remediation, execution transparency is paramount: Logistic Regression provides exact, closed-form linear attribution ($w_i \cdot \tilde{x}_i$) without requiring sampling-based approximations (such as Kernel-SHAP or surrogate LIME models). When operators receive an automated recommendation, individual feature contributions and resource evidence scores can be audited deterministically with zero runtime approximation latency.

C. Attribution Boundaries and Verification Honesty

Aggregating attribution scores across resource domains reliably isolates primary fault categories for gradual failure modes (CPU, memory, disk). However, this heuristic approach cannot establish counterfactual causal relationships and exhibits lower diagnostic resolution on compound, multi-resource failures. Furthermore, the post-recovery verification mechanism conservatively returned an `INCONCLUSIVE` verdict during network degradation evaluation (risk reduced from 0.884 to 0.447). Rather than overstating recovery success, the tripartite verdict structure (`RECOVERED`, `NOT_RECOVERED`, `INCONCLUSIVE`) enforces essential academic and operational honesty.

VII. SYSTEM LIMITATIONS AND THREATS TO VALIDITY

Key constraints must be explicitly acknowledged: (1) **Synthetic Telemetry:** Telemetry is generated via parameterised stochastic distributions; validation on physical multi-tenant cloud hardware remains necessary. (2) **Simulated Remediation:** Recovery operations are logged mock directives rather than destructive hypervisor or cloud provider API calls. (3) **Sudden Failure Bounds:** Abrupt step-function spikes exhibit lower predictive sensitivity (F1 0.529) due to the absence of precursor trend signals. (4) **Non-Causal Attribution:** Resource-level evidence aggregation reflects model focus rather than formal counterfactual causal mechanisms. (5) **Synthetic Recovery Proxy:** Post-healing verification evaluates restored synthetic normal streams rather than physical platform convalescence. (6) **Runtime Initialisation:** The research prototype fits the model at server startup, which is appropriate for evaluation but requires dedicated model registries in production. (7) **Empirical Scope:** Benchmarking against production traces (e.g., Google or Alibaba cluster datasets) represents crucial future validation.

VIII. CONCLUSION AND FUTURE WORK

This paper demonstrated an end-to-end, explainable autonomous cloud management platform integrating ten lifecycle stages from telemetry ingestion through post-remediation verification. Experimental evaluation confirms: (1) Isolation Forest achieves F1 0.760 on snapshot telemetry; (2) temporal Logistic Regression attains mean F1 0.692 and ROC-AUC

0.813, substantially exceeding snapshot threshold baselines (F1 0.156); (3) temporal rolling features deliver an advance early-warning lead time averaging 18.06 timesteps prior to failure onset; (4) deterministic coefficient attribution isolates influential slope and variability dynamics without sampling approximations; and (5) the closed-loop verification engine enforces an honest, conservative standard by reporting INCONCLUSIVE on partial remediations.

Future work will transition this framework to physical cloud testbeds, integrate real orchestration APIs (e.g., Kubernetes operators), examine multi-model attribution consensus, and benchmark deep sequence architectures against this interpretable linear baseline.

ACKNOWLEDGMENT

The author expresses sincere gratitude to project guide Mrs. S. Shirly, Assistant Professor, Department of Computer Science and Engineering, Karunya Institute of Technology and Science, for her invaluable supervision, guidance, and constructive feedback throughout this research. In compliance with IEEE author guidelines, the author acknowledges the use of AI tools for assistance with manuscript language refinement and formatting; all technical concepts, experimental analyses, and conclusions were independently conducted, verified, and approved by the author.

REFERENCES

[1] Q. Lin, H. Zhang, J. Lou, Y. Zhang, and X. Liu, "AIOps: Real-world challenges and research innovations," in *Proceedings of the 41st International Conference on Software Engineering: Companion (ICSE-Companion)*, 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8802619

[2] J. Bogatinovski, S. Nedelkoski, J. Cardoso, and O. Kao, "Artificial intelligence for IT operations (AIOps) on cloud platforms," 2022. [Online]. Available: https://arxiv.org/abs/2204.11422

[3] C. Niu and et al., "Deep learning for IT operations (AIOps): A survey," 2020. [Online]. Available: https://arxiv.org/abs/2001.03741

[4] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," 2017. [Online]. Available: https://dl.acm.org/doi/10.1145/3133956.3134015

[5] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, and Y. Chen, "LogAnomaly: Unsupervised detection of sequential anomalies using deep learning," in *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, 2019. [Online]. Available: https://www.ijcai.org/Proceedings/2019/658

[6] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019. [Online]. Available: https://dl.acm.org/doi/10.1145/3292500.3330672

[7] H. Xu, W. Chen, N. Zhao, Z. Li, J. Bu, Z. Li, Y. Liu, Y. Zhao, and D. Pei, "Unsupervised anomaly detection via variational auto-encoder for seasonal KPIs in web applications," in *Proceedings of the 2018 World Wide Web Conference (WWW)*, 2018. [Online]. Available: https://dl.acm.org/doi/10.1145/3178876.3185996

[8] A. Samir, C. Dehury, and P. Harsh, "Self-healing cloud microservices using advanced fuzzy logic," *IEEE Access*, 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8880711

[9] H. Zhang and et al., "Towards automated remediation for cloud outages," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2022. [Online]. Available: https://dl.acm.org/doi/10.1145/3540250.3549117

[10] P. Wang, J. Sun, H. Wang, J. Wang, Q. Qi, and J. Liao, "CloudRanger: Root cause identification for cloud native systems," in *Proceedings of the 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*, 2018. [Online]. Available: https://ieeexplore.ieee.org/document/8411019

[11] J. Weng, J. Wang, J. Yang, and Y. Yang, "Root cause analysis of anomalies of multitier services in public clouds," in *Proceedings of the 26th International Symposium on Quality of Service (IWQoS)*, 2018. [Online]. Available: https://ieeexplore.ieee.org/document/8624177

[12] J. Chen and et al., "CauseInfer: Automated end-to-end performance diagnosis with hierarchical causality graph in large distributed systems," in *Proceedings of the 33rd IEEE International Conference on Computer Communications (INFOCOM)*, 2014. [Online]. Available: https://ieeexplore.ieee.org/document/6847987

[13] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis for anomaly detection," in *Proceedings of the 27th IEEE International Symposium on Software Reliability Engineering (ISSRE)*, 2016. [Online]. Available: https://ieeexplore.ieee.org/document/7742964

[14] Z. Li, Y. Zhao, J. Han, Y. Su, R. Jiao, X. Wen, and D. Pei, "InterFusion: Interacting hidden dynamics and macroscopic observations for multivariate time series anomaly detection," in *Proceedings of the 27th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2021. [Online]. Available: https://dl.acm.org/doi/10.1145/3447548.3467077

[15] S. Nedelkoski, J. Bogatinovski, A. Acker, J. Cardoso, and O. Kao, "Anomaly detection and classification using distributed tracing and deep learning," in *Proceedings of the 19th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*, 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8752864

[16] M. M. Islam and et al., "A systematic literature review of cloud monitoring tools," *Journal of Network and Computer Applications*, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1084804519300104

[17] M. T. Ribeiro, S. Singh, and C. Guestrin, ""why should I trust you?": Explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016. [Online]. Available: https://dl.acm.org/doi/10.1145/2939672.2939778

[18] S. M. Lundberg and S. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017. [Online]. Available: https://arxiv.org/abs/1705.07874

[19] Z. Li, Y. Shu, Y. Gong, and Y. Zhao, "A survey on explainable anomaly detection," 2023. [Online]. Available: https://arxiv.org/abs/2209.10398

[20] Y. Gan, Y. Zhang, D. Nguyen, T. Mahindra, and et al., "Seer: Leveraging big data to navigate the complexity of cloud-based debugging," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019. [Online]. Available: https://dl.acm.org/doi/10.1145/3297858.3304004